Algoritmi di ordinamento

Indice

Introduzione

Il problema dell'ordinamento

Insertion Sort

Exchange Sort

Selection Sort

Confronto dei tre algoritmi

Introduzione

Introduzione: l'ordinamento

- L'ordinamento è uno dei compiti più comuni dell'informatica
- Un elenco ordinato di nomi, di voti o di prezzi è molto più facile da **consultare**, da **confrontare** e da **leggere** di uno non ordinato

Che cosa facciamo in questo capitolo

- Enunciamo con precisione il **problema dell'ordinamento**
- Introduciamo i **criteri** con cui si confrontano gli algoritmi di ordinamento
- Studiamo tre algoritmi classici che lo risolvono:
  - **Insertion Sort**
  - **Exchange Sort**
  - **Selection Sort**

Come studiamo i tre algoritmi

- Sono scritti nello pseudocodice del corso, con gli array indicizzati a partire da 1
- Per ogni algoritmo mostriamo l'esecuzione passo per passo sugli stessi due array,

$$[5, 2, 4, 1, 3] \qquad \text{e} \qquad [12, 11, 13, 5, 6],$$

  in modo che i tre metodi possano essere confrontati direttamente
- Per ogni algoritmo contiamo i confronti tra chiavi e gli spostamenti di chiavi che esegue, e stabiliamo se è in-place e se è stabile

Il problema dell'ordinamento

Introduzione: Il problema dell'ordinamento

- In questa sezione definiamo il problema dell'ordinamento: input, output, e che cosa vuol dire "ordinato"
- Fissiamo le sole operazioni che un algoritmo basato su confronti può eseguire sulle chiavi
- Introduciamo i criteri con cui confrontiamo gli algoritmi di ordinamento: confronti, assegnazioni, memoria aggiuntiva e stabilità
- Questi criteri sono il metro di misura per i tre algoritmi del capitolo

Definizione 1: problema dell'ordinamento

Dato un array $a[1..n]$ *di* $n \in \mathbb{Z}_{\geq 1}$ *numeri naturali, il* ***problema dell'ordinamento*** *consiste nel riarrangiare gli elementi dell'array in* ***ordine non decrescente****, cioè in modo che*

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

Un ***riarrangiamento*** *(o* ***permutazione****) dell'array è un array che contiene gli stessi elementi, ciascuno con la stessa molteplicità, in un ordine eventualmente diverso. L'output del problema dell'ordinamento è il riarrangiamento dell'array di input che soddisfa la condizione precedente.*

Chiavi e ordine non decrescente

- Ogni elemento $a[i]$ è detto **chiave**: è il valore su cui si basa l'ordinamento, cioè il valore che viene confrontato per decidere la disposizione finale
- L'ordine è *non decrescente* e non *crescente* perché l'array può contenere chiavi uguali, come in $[3, 1, 3]$, il cui riarrangiamento ordinato è $[1, 3, 3]$
- L'output deve essere un riarrangiamento dell'input: un array come $[0, 0, 0, 0, 0]$ soddisfa $a[1] \leq a[2] \leq \cdots \leq a[n]$, ma non è la versione ordinata di $[5, 2, 4, 1, 3]$

Esempio

Ordinamento di un array

- Array di input: $a = [5, 2, 4, 1, 3]$
- Array ordinato: $a = [1, 2, 3, 4, 5]$

Prima di studiare algoritmi specifici,
fissiamo le operazioni che essi possono usare
e i criteri con cui li confrontiamo.

Ordinamento basato su confronti

Definizione 2: ordinamento basato su confronti

*Un algoritmo di ordinamento è **basato su confronti** se accede alle chiavi soltanto confrontandole a due a due e copiandole.*

- Tutti gli algoritmi di ordinamento studiati in questo capitolo (Insertion Sort, Exchange Sort, Selection Sort) appartengono a questa classe: decidono come riarrangiare gli elementi soltanto in base ai confronti tra coppie di chiavi

Operazioni sulle chiavi

- Le sole operazioni consentite sulle chiavi in un algoritmo di ordinamento basato su confronti sono:
  1. **confrontare due chiavi**: stabilire quale tra $a[i]$ e $a[j]$ è la maggiore (o se sono uguali);
  2. **copiare / assegnare una chiave**: spostare o memorizzare il valore di una chiave, ad esempio $tmp \leftarrow a[i]$

- Operazioni vietate sulle chiavi:
  1. operazioni aritmetiche sulle chiavi (addizione, moltiplicazione, ecc.);
  2. ispezionare le singole cifre o i singoli bit di una chiave;
  3. qualunque operazione che sfrutti la rappresentazione interna della chiave

Metriche di prestazione (1/2)

- Poiché i confronti e le assegnazioni sono le sole operazioni consentite sulle chiavi, valutiamo un algoritmo di ordinamento basato su confronti contando:
  - il **numero di confronti tra chiavi** che esegue;
  - il **numero di assegnazioni di chiavi** (copie) che esegue

- Il numero di assegnazioni conta nella pratica. Ogni **scambio** (swap) di due elementi dell'array richiede tre assegnazioni:

$$tmp \leftarrow a[i]; \qquad a[i] \leftarrow a[j]; \qquad a[j] \leftarrow tmp,$$

quindi gli algoritmi che scambiano meno tendono a essere più veloci

Metriche di prestazione (2/2)

- Entrambi i numeri dipendono non solo da n ma anche dall'ordine iniziale delle chiavi
- Per questo motivo li calcoliamo nel **caso migliore** (l'input che li minimizza) e nel **caso peggiore** (l'input che li massimizza), tra tutti gli array di n elementi
- Nei conteggi usiamo la somma dei primi $n-1$ interi positivi:

$$\sum_{k=1}^{n-1} k \;=\; 1+2+\cdots+(n-1) \;=\; \frac{n(n-1)}{2}$$

Ordinamento in-place e uso della memoria

- Oltre al tempo, ci interessa anche quanta **memoria aggiuntiva** usa un algoritmo, oltre allo spazio già necessario per memorizzare le n chiavi di input

Definizione 3: ordinamento in-place

*Un algoritmo di ordinamento è **in-place** (in loco) se la quantità di memoria aggiuntiva che richiede è **costante**, cioè non cresce con n.*

Algoritmi in-place

- In modo equivalente, un algoritmo di ordinamento in-place ordina riarrangiando gli elementi *all'interno* dell'array originale, usando al più un numero fisso di variabili ausiliarie (ad esempio, i contatori dei cicli e una variabile temporanea per lo scambio)

- Gli algoritmi in-place riarrangiano tipicamente l'array di input spostando o scambiando elementi, quindi non richiedono spazio aggiuntivo per un altro array che contenga l'output ordinato

Stabilità

- In molte applicazioni le chiavi fanno parte di **record** più grandi (ad esempio, uno studente identificato da un voto e da un nome), e due record diversi possono avere la stessa chiave
- Scriviamo un record come una coppia (*key*, *label*). Si confrontano solo le chiavi, e ogni etichetta viaggia insieme alla sua chiave

Definizione 4: ordinamento stabile

*Un algoritmo di ordinamento è **stabile** se i record con chiavi uguali compaiono nell'output nello stesso ordine relativo in cui compaiono nell'input.*

Esempio

Ordinamento stabile e non stabile

- Array di record: $a = [(3, A),\ (1, B),\ (3, C),\ (2, D),\ (1, E)]$
- Ordinamento stabile:
  $a = [(1, B),\ (1, E),\ (2, D),\ (3, A),\ (3, C)]$
  - I record con chiave 1 mantengono l'ordine B, E e i record con chiave 3 mantengono l'ordine A, C, come nell'input
- Ordinamento non stabile:
  $a = [(1, E),\ (1, B),\ (2, D),\ (3, C),\ (3, A)]$
  - Le chiavi sono ordinate, ma entrambe le coppie di record con chiavi uguali risultano invertite

Perché la stabilità conta

- La stabilità conta quando i record vengono ordinati più volte rispetto a chiavi diverse
- Ad esempio, ordinando gli studenti prima per nome e poi, con un algoritmo stabile, per voto, gli studenti con lo stesso voto risultano in ordine alfabetico

Come si valuta un algoritmo di ordinamento

Un algoritmo di ordinamento basato su confronti si valuta in base

- al **numero di confronti tra chiavi** e al **numero di assegnazioni di chiavi** che esegue (nel caso migliore e nel caso peggiore)
- alla **memoria aggiuntiva** che usa (in-place o no)
- al fatto di essere **stabile**

Riepilogo

Il problema dell'ordinamento:

- Riarrangiare $a[1..n]$ in modo che $a[1] \leq a[2] \leq \cdots \leq a[n]$; l'output è un riarrangiamento dell'input

Ordinamento basato su confronti:

- Le chiavi vengono soltanto confrontate a due a due e copiate
- Si valuta con i confronti tra chiavi e le assegnazioni di chiavi, nel caso migliore e peggiore; uno scambio costa tre assegnazioni

Memoria e stabilità:

- In-place: memoria aggiuntiva costante
- Stabile: le chiavi uguali mantengono il loro ordine relativo

Insertion Sort

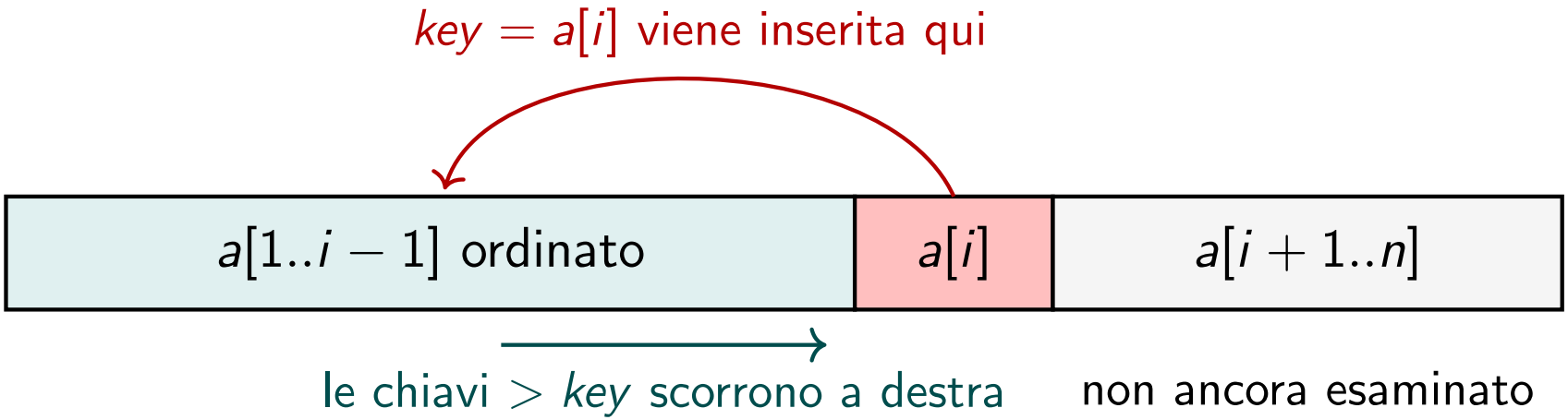
Introduzione: Insertion Sort

- Il primo algoritmo: costruisce un prefisso ordinato dell'array e inserisce ogni nuova chiave al suo posto
- Diamo il suo pseudocodice e lo eseguiamo passo per passo sui due array del capitolo
- Contiamo i suoi confronti e spostamenti, e mostriamo che è in-place e stabile
- È l'unico algoritmo del capitolo che si adatta all'input: su un array ordinato lavora pochissimo

L'idea di Insertion Sort

- Insertion Sort funziona come molte persone ordinano una mano di carte da gioco: le carte già in mano si tengono ordinate, e ogni nuova carta viene inserita al suo posto tra di esse
- All'inizio dell'iterazione i, il sotto-array $a[1..i-1]$ è ordinato
- L'algoritmo prende l'elemento successivo $a[i]$, detto *key*, sposta di una posizione a destra tutti gli elementi di $a[1..i-1]$ maggiori di *key* e scrive *key* nella posizione rimasta libera

Iterazione i di Insertion Sort



- $key = a[i]$ viene inserita nel sotto-array ordinato $a[1..i-1]$, dopo aver spostato di una posizione a destra le chiavi maggiori di key

Algorithm: InsertionSort (1/2)

Input : Un array $a[1..n]$ di n numeri naturali
Output : L'array a ordinato in ordine non decrescente

```
1 function InsertionSort(a, n)
2     i := 2
3     while i ≤ n do
4         key := a[i]                    // elemento da inserire
5         j := i − 1
6         while j ≥ 1 and a[j] > key do
7             a[j + 1] ← a[j]            // sposta l'elemento a
                destra
8             j ← j − 1
9         end
```

Algorithm: InsertionSort (2/2)

```
function InsertionSort(a, n) (continua)
    while i ≤ n (continua) do
10      a[j + 1] ← key        // inserisce key nella sua
         posizione
11      i ← i + 1
12  end
13 end
```

Esegui in Colab: Python · C++ · Java

Come funziona InsertionSort (1/2)

- Il ciclo esterno (righe 3–11) scorre l'array a partire dal secondo elemento ($i = 2$): un singolo elemento, $a[1..1]$, è già ordinato
- Per ogni elemento $a[i]$ (la *key*), il ciclo interno (righe 6–8) scorre la parte ordinata $a[1..i-1]$ da destra a sinistra e sposta di una posizione a destra tutti gli elementi maggiori di *key*

Come funziona InsertionSort (2/2)

- Il ciclo interno si ferma quando $j = 0$ (tutti gli elementi di $a[1..i-1]$ erano maggiori di *key*) oppure quando $a[j] \leq key$
- La condizione alla riga 6 è valutata da sinistra a destra: quando $j = 0$ il confronto $a[j] > key$ non viene valutato, perché $a[0]$ non esiste
- La riga 10 scrive *key* nella posizione $j + 1$ lasciata libera dagli spostamenti
- Dopo ogni iterazione del ciclo esterno, il sotto-array $a[1..i]$ è ordinato. Quando l'algoritmo termina, $i = n + 1$ e l'intero array $a[1..n]$ è ordinato

Come leggere le tabelle di traccia

- Nelle tabelle di traccia di questo capitolo, ogni riga mostra lo stato dell'array *dopo* l'operazione descritta nell'ultima colonna
- La chiave corrente è evidenziata in rosso
- Le celle modificate da uno spostamento, da un inserimento o da uno scambio sono evidenziate in blu

Esempio: Insertion Sort su $[5, 2, 4, 1, 3]$ (1/3)

Array di input: $a = [5, 2, 4, 1, 3]$, con $n = 5$. Il ciclo esterno esegue le iterazioni $i = 2, 3, 4, 5$.

Traccia dettagliata (ciclo interno), iterazioni $i = 2$ e $i = 3$:

i	j	*key*	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	–	–	5	2	4	1	3	stato iniziale
2	1	2	5	2	4	1	3	$key \leftarrow a[2] = 2$; $j \leftarrow 1$
	0	2	5	5	4	1	3	$a[1] = 5 > 2$: sposta $a[2] \leftarrow 5$; $j \leftarrow 0$
	–	2	2	5	4	1	3	$j = 0$: esci; inserisci $a[1] \leftarrow 2$
3	2	4	2	5	4	1	3	$key \leftarrow a[3] = 4$; $j \leftarrow 2$
	1	4	2	5	5	1	3	$a[2] = 5 > 4$: sposta $a[3] \leftarrow 5$; $j \leftarrow 1$
	–	4	2	4	5	1	3	$a[1] = 2 \leq 4$: esci; inserisci $a[2] \leftarrow 4$

Esempio: Insertion Sort su $[5, 2, 4, 1, 3]$ (2/3)

Traccia dettagliata (ciclo interno), iterazioni $i = 4$ e $i = 5$:

i	j	*key*	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
4	3	1	2	4	5	1	3	$key \leftarrow a[4] = 1$; $j \leftarrow 3$
	2	1	2	4	5	5	3	$a[3] = 5 > 1$: sposta $a[4] \leftarrow 5$; $j \leftarrow 2$
	1	1	2	4	4	5	3	$a[2] = 4 > 1$: sposta $a[3] \leftarrow 4$; $j \leftarrow 1$
	0	1	2	2	4	5	3	$a[1] = 2 > 1$: sposta $a[2] \leftarrow 2$; $j \leftarrow 0$
	–	1	1	2	4	5	3	$j = 0$: esci; inserisci $a[1] \leftarrow 1$
5	4	3	1	2	4	5	3	$key \leftarrow a[5] = 3$; $j \leftarrow 4$
	3	3	1	2	4	5	5	$a[4] = 5 > 3$: sposta $a[5] \leftarrow 5$; $j \leftarrow 3$
	2	3	1	2	4	4	5	$a[3] = 4 > 3$: sposta $a[4] \leftarrow 4$; $j \leftarrow 2$
	–	3	1	2	3	4	5	$a[2] = 2 \leq 3$: esci; inserisci $a[3] \leftarrow 3$

Esempio: Insertion Sort su $[5, 2, 4, 1, 3]$ (3/3)

Stato dell'array dopo ogni iterazione del ciclo esterno:

i	key	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	–	5	2	4	1	3	stato iniziale
2	2	2	5	4	1	3	inserisci 2 all'inizio
3	4	2	4	5	1	3	inserisci 4 tra 2 e 5
4	1	1	2	4	5	3	inserisci 1 all'inizio
5	3	1	2	3	4	5	inserisci 3 tra 2 e 4

- L'array ordinato è $[1, 2, 3, 4, 5]$
- L'algoritmo esegue 9 confronti tra chiavi e 7 spostamenti

Esempio: Insertion Sort su $[12, 11, 13, 5, 6]$ (1/3)

Array di input: $a = [12, 11, 13, 5, 6]$, con $n = 5$.

Traccia dettagliata (ciclo interno), iterazioni $i = 2$ e $i = 3$:

i	j	*key*	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	–	–	12	11	13	5	6	stato iniziale
2	1	11	12	11	13	5	6	$key \leftarrow a[2] = 11$; $j \leftarrow 1$
	0	11	12	12	13	5	6	$a[1] = 12 > 11$: sposta $a[2] \leftarrow 12$; $j \leftarrow 0$
	–	11	11	12	13	5	6	$j = 0$: esci; inserisci $a[1] \leftarrow 11$
3	2	13	11	12	13	5	6	$key \leftarrow a[3] = 13$; $j \leftarrow 2$
	–	13	11	12	13	5	6	$a[2] = 12 \leq 13$: esci; inserisci $a[3] \leftarrow 13$

Esempio: Insertion Sort su $[12, 11, 13, 5, 6]$ (2/3)

Traccia dettagliata (ciclo interno), iterazioni $i = 4$ e $i = 5$:

i	j	*key*	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
4	3	5	11	12	13	5	6	$key \leftarrow a[4] = 5$; $j \leftarrow 3$
	2	5	11	12	13	13	6	$a[3] = 13 > 5$: sposta $a[4] \leftarrow 13$; $j \leftarrow 2$
	1	5	11	12	12	13	6	$a[2] = 12 > 5$: sposta $a[3] \leftarrow 12$; $j \leftarrow 1$
	0	5	11	11	12	13	6	$a[1] = 11 > 5$: sposta $a[2] \leftarrow 11$; $j \leftarrow 0$
	–	5	5	11	12	13	6	$j = 0$: esci; inserisci $a[1] \leftarrow 5$
5	4	6	5	11	12	13	6	$key \leftarrow a[5] = 6$; $j \leftarrow 4$
	3	6	5	11	12	13	13	$a[4] = 13 > 6$: sposta $a[5] \leftarrow 13$; $j \leftarrow 3$
	2	6	5	11	12	12	13	$a[3] = 12 > 6$: sposta $a[4] \leftarrow 12$; $j \leftarrow 2$
	1	6	5	11	11	12	13	$a[2] = 11 > 6$: sposta $a[3] \leftarrow 11$; $j \leftarrow 1$
	–	6	5	6	11	12	13	$a[1] = 5 \leq 6$: esci; inserisci $a[2] \leftarrow 6$

Esempio: Insertion Sort su $[12, 11, 13, 5, 6]$ (3/3)

Stato dell'array dopo ogni iterazione del ciclo esterno:

i	key	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	–	12	11	13	5	6	stato iniziale
2	11	11	12	13	5	6	inserisci 11 all'inizio
3	13	11	12	13	5	6	13 già al suo posto
4	5	5	11	12	13	6	inserisci 5 all'inizio
5	6	5	6	11	12	13	inserisci 6 tra 5 e 11

- L'array ordinato è $[5, 6, 11, 12, 13]$
- Nell'iterazione $i = 3$ la chiave 13 è già al suo posto: il ciclo interno si ferma al primo confronto e non viene eseguito alcuno spostamento
- L'algoritmo esegue 9 confronti tra chiavi e 7 spostamenti

Proprietà di Insertion Sort

- I due esempi mostrano che il lavoro svolto da Insertion Sort dipende dall'ordine iniziale delle chiavi
- Una chiave già al suo posto costa un confronto e nessuno spostamento
- Una chiave più piccola di tutte le precedenti viene spostata lungo l'intera parte ordinata

Osservazione 1: confronti e spostamenti di Insertion Sort

Dato un array $a[1..n]$ di n numeri naturali, `InsertionSort` esegue almeno $n-1$ e al più $\frac{n(n-1)}{2}$ confronti tra chiavi, e al più $\frac{n(n-1)}{2}$ spostamenti. Le limitazioni inferiori sono raggiunte quando l'array è già ordinato (non viene eseguito alcuno spostamento), quelle superiori quando è ordinato in senso decrescente con chiavi distinte.

Dimostrazione (1/3)

- Consideriamo l'iterazione i del ciclo esterno, con $i \in \{2, 3, \ldots, n\}$, e siano s_i e c_i il numero di spostamenti e di confronti tra chiavi che essa esegue
- Il ciclo interno parte con $j = i - 1 \geq 1$, quindi $c_i \geq 1$
- Ogni spostamento (righe 7–8) segue un confronto $a[j] > key$ vero e diminuisce j di 1; quindi $s_i \leq i - 1$

Dimostrazione (2/3)

Il ciclo interno si ferma poi in uno di due modi:

- con $j = 0$, dopo $s_i = i - 1$ spostamenti e senza un ulteriore confronto: $c_i = s_i = i - 1$;
- dopo un confronto $a[j] > key$ falso, con $j \geq 1$: allora $s_i = i - 1 - j \leq i - 2$ e $c_i = s_i + 1 \leq i - 1$

In entrambi i casi $1 \leq c_i \leq i - 1$ e $0 \leq s_i \leq i - 1$. Sommando su $i \in \{2, 3, \ldots, n\}$ e usando la somma dei primi $n - 1$ interi positivi:

$$n - 1 \;\leq\; \text{confronti tra chiavi} \;\leq\; \sum_{i=2}^{n} (i - 1) = \frac{n(n-1)}{2},$$

$$\text{spostamenti} \;\leq\; \frac{n(n-1)}{2}$$

Dimostrazione (3/3)

- Se l'array è ordinato, a ogni iterazione $a[i-1] \leq key$: il ciclo interno si ferma al primo confronto, senza spostamenti, e l'algoritmo esegue esattamente $n-1$ confronti
- Se le chiavi sono distinte e in ordine decrescente, $key = a[i]$ è minore di tutte le chiavi di $a[1..i-1]$: vengono spostate tutte e il ciclo si ferma con $j = 0$
- Quindi l'iterazione i esegue $i-1$ confronti e $i-1$ spostamenti, e i totali sono entrambi $\frac{n(n-1)}{2}$

□

Assegnazioni di Insertion Sort

- Ogni iterazione esegue inoltre due assegnazioni di chiavi (righe 4 e 10), quindi il numero di assegnazioni di chiavi è $2(n-1)$ più il numero di spostamenti: tra

$$2(n-1) \qquad \text{e} \qquad 2(n-1) + \frac{n(n-1)}{2}$$

- Insertion Sort non scambia mai due elementi: uno spostamento è una singola assegnazione, il che rende il metodo più economico di una sequenza di scambi che muoverebbe *key* di una posizione alla volta

Osservazione 2: Insertion Sort è in-place e stabile

`InsertionSort` è un algoritmo di ordinamento in-place e stabile.

Dimostrazione

- Oltre all'array, l'algoritmo usa soltanto le tre variabili i, j e *key*, indipendentemente da n: è in-place
- Una chiave viene spostata a sinistra soltanto oltre chiavi *strettamente* maggiori di essa (condizione $a[j] > key$ alla riga 6)
- Il ciclo interno si ferma alla prima chiave $a[j] \leq key$, quindi *key* viene inserita dopo ogni chiave uguale che la precede
- Dunque due record con chiavi uguali non vengono mai scambiati, e il loro ordine relativo nell'output è lo stesso dell'input: l'algoritmo è stabile

□

Dallo pseudocodice al codice

- Nelle implementazioni (C++, Python e Java, nei notebook Colab qui sopra) l'array ha indici da 0 a $n - 1$
- Il ciclo esterno parte da $i = 1$, e il ciclo interno si ferma quando j diventa -1
- In C++ l'operatore `&&` valuta il suo operando destro solo quando quello sinistro è vero, quindi `a[j]` non viene mai letto con $j = -1$

Riepilogo

Insertion Sort:

- Tiene ordinato $a[1..i-1]$ e inserisce $key = a[i]$ dopo aver spostato a destra le chiavi maggiori di essa

Costo:

- Tra $n-1$ (array ordinato) e $\frac{n(n-1)}{2}$ (array decrescente) confronti tra chiavi
- Al più $\frac{n(n-1)}{2}$ spostamenti, di una assegnazione ciascuno; nessuno scambio

Proprietà:

- In-place e stabile

Exchange Sort

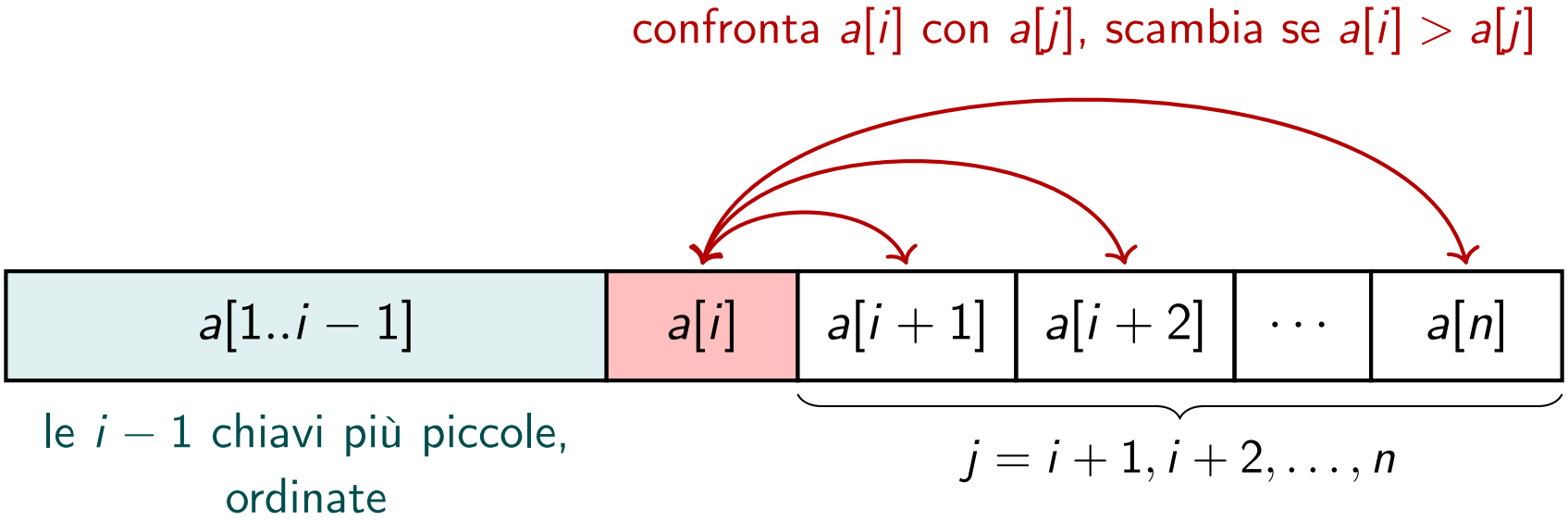
Introduzione: Exchange Sort

- Il secondo algoritmo segue l'idea opposta: riempie le posizioni una alla volta, ciascuna con la più piccola tra le chiavi rimaste
- Diamo il suo pseudocodice e lo eseguiamo sugli stessi due array
- Vedremo che esegue sempre lo stesso numero di confronti, ma può scambiare molte volte, e che non è stabile

L'idea di Exchange Sort

- Insertion Sort costruisce il prefisso ordinato inserendo ogni nuova chiave tra le precedenti
- Exchange Sort segue l'idea opposta: riempie le posizioni $1, 2, \ldots, n-1$ una alla volta, ciascuna con la più piccola tra le chiavi rimaste
- Nell'iterazione i, l'algoritmo confronta $a[i]$ con ogni elemento $a[j]$ con $j > i$ e scambia i due elementi ogni volta che $a[i] > a[j]$
- Alla fine dell'iterazione $a[i]$ è il minimo di $a[i..n]$

Iterazione i di Exchange Sort



- $a[i]$ viene confrontato con ogni $a[j]$, $j > i$, e i due elementi vengono scambiati ogni volta che $a[i] > a[j]$; alla fine, $a[i] = \min(a[i..n])$

Algorithm: ExchangeSort (1/2)

Input : Un array $a[1..n]$ di n numeri naturali
Output : L'array a ordinato in ordine non decrescente

```
1  function ExchangeSort(a, n)
2      i := 1
3      while i ≤ n − 1 do
4          j := i + 1
5          while j ≤ n do
6              if a[i] > a[j] then
7                  tmp := a[i]            // scambia a[i] e a[j]
8                  a[i] ← a[j]
9                  a[j] ← tmp
10             end
11             j ← j + 1
12         end
```

Algorithm: ExchangeSort (2/2)

```
function ExchangeSort(a, n) (continua)
    while i ≤ n − 1 (continua) do
13      i ← i + 1
14  end
15 end
```

Esegui in Colab: Python · C++ · Java

Come funziona ExchangeSort

- Il ciclo esterno scorre con l'indice i da 1 a $n-1$
- Per ogni i, il ciclo interno confronta $a[i]$ con ogni elemento $a[j]$ per $j \in \{i+1, i+2, \ldots, n\}$
- Ogni volta che $a[i] > a[j]$, i due elementi vengono scambiati (righe 7–9), quindi $a[i]$ contiene sempre la più piccola chiave incontrata finora in $a[i..n]$
- Dopo l'iterazione i del ciclo esterno, $a[i]$ è il minimo di $a[i..n]$ e il sotto-array $a[1..i]$ contiene le i chiavi più piccole dell'array, ordinate
- Dopo l'iterazione $n-1$, l'ultimo elemento $a[n]$ è la chiave maggiore, e l'intero array è ordinato

Esempio: Exchange Sort su $[5, 2, 4, 1, 3]$ (1/2)

Array di input: $a = [5, 2, 4, 1, 3]$, con $n = 5$. Il ciclo esterno esegue le iterazioni $i = 1, 2, 3, 4$. Traccia dettagliata (ciclo interno):

i	j	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	–	5	2	4	1	3	stato iniziale
1	2	2	5	4	1	3	$a[1] = 5 > a[2] = 2$: scambia $a[1] \leftrightarrow a[2]$
	3	2	5	4	1	3	$a[1] = 2 \leq a[3] = 4$: nessuno scambio
	4	1	5	4	2	3	$a[1] = 2 > a[4] = 1$: scambia $a[1] \leftrightarrow a[4]$
	5	1	5	4	2	3	$a[1] = 1 \leq a[5] = 3$: nessuno scambio
2	3	1	4	5	2	3	$a[2] = 5 > a[3] = 4$: scambia $a[2] \leftrightarrow a[3]$
	4	1	2	5	4	3	$a[2] = 4 > a[4] = 2$: scambia $a[2] \leftrightarrow a[4]$
	5	1	2	5	4	3	$a[2] = 2 \leq a[5] = 3$: nessuno scambio
3	4	1	2	4	5	3	$a[3] = 5 > a[4] = 4$: scambia $a[3] \leftrightarrow a[4]$
	5	1	2	3	5	4	$a[3] = 4 > a[5] = 3$: scambia $a[3] \leftrightarrow a[5]$
4	5	1	2	3	4	5	$a[4] = 5 > a[5] = 4$: scambia $a[4] \leftrightarrow a[5]$

Esempio: Exchange Sort su $[5, 2, 4, 1, 3]$ (2/2)

Stato dell'array dopo ogni iterazione del ciclo esterno:

i	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	5	2	4	1	3	stato iniziale
1	1	5	4	2	3	$a[1] = \min(a[1..5]) = 1$ (2 scambi)
2	1	2	5	4	3	$a[2] = \min(a[2..5]) = 2$ (2 scambi)
3	1	2	3	5	4	$a[3] = \min(a[3..5]) = 3$ (2 scambi)
4	1	2	3	4	5	$a[4] = \min(a[4..5]) = 4$ (1 scambio)

- L'array ordinato è $[1, 2, 3, 4, 5]$
- L'algoritmo esegue 10 confronti tra chiavi e 7 scambi

Esempio: Exchange Sort su $[12, 11, 13, 5, 6]$ (1/2)

Array di input: $a = [12, 11, 13, 5, 6]$, con $n = 5$. Traccia dettagliata (ciclo interno):

i	j	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	–	12	11	13	5	6	stato iniziale
1	2	11	12	13	5	6	$a[1] = 12 > a[2] = 11$: scambia $a[1] \leftrightarrow a[2]$
	3	11	12	13	5	6	$a[1] = 11 \leq a[3] = 13$: nessuno scambio
	4	5	12	13	11	6	$a[1] = 11 > a[4] = 5$: scambia $a[1] \leftrightarrow a[4]$
	5	5	12	13	11	6	$a[1] = 5 \leq a[5] = 6$: nessuno scambio
2	3	5	12	13	11	6	$a[2] = 12 \leq a[3] = 13$: nessuno scambio
	4	5	11	13	12	6	$a[2] = 12 > a[4] = 11$: scambia $a[2] \leftrightarrow a[4]$
	5	5	6	13	12	11	$a[2] = 11 > a[5] = 6$: scambia $a[2] \leftrightarrow a[5]$
3	4	5	6	12	13	11	$a[3] = 13 > a[4] = 12$: scambia $a[3] \leftrightarrow a[4]$
	5	5	6	11	13	12	$a[3] = 12 > a[5] = 11$: scambia $a[3] \leftrightarrow a[5]$
4	5	5	6	11	12	13	$a[4] = 13 > a[5] = 12$: scambia $a[4] \leftrightarrow a[5]$

Esempio: Exchange Sort su $[12, 11, 13, 5, 6]$ (2/2)

Stato dell'array dopo ogni iterazione del ciclo esterno:

i	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	12	11	13	5	6	stato iniziale
1	5	12	13	11	6	$a[1] = \min(a[1..5]) = 5$ (2 scambi)
2	5	6	13	12	11	$a[2] = \min(a[2..5]) = 6$ (2 scambi)
3	5	6	11	13	12	$a[3] = \min(a[3..5]) = 11$ (2 scambi)
4	5	6	11	12	13	$a[4] = \min(a[4..5]) = 12$ (1 scambio)

- L'array ordinato è $[5, 6, 11, 12, 13]$
- L'algoritmo esegue 10 confronti tra chiavi e 7 scambi

Proprietà di Exchange Sort

- A differenza di Insertion Sort, Exchange Sort confronta ogni coppia di posizioni $i < j$ esattamente una volta, qualunque sia l'ordine iniziale delle chiavi
- Solo il numero di scambi dipende dall'input

Osservazione 3: confronti e scambi di Exchange Sort

Dato un array $a[1..n]$ di n numeri naturali, `ExchangeSort` esegue esattamente $\frac{n(n-1)}{2}$ confronti tra chiavi e al più $\frac{n(n-1)}{2}$ scambi. Non viene eseguito alcuno scambio quando l'array è già ordinato, e vengono eseguiti $\frac{n(n-1)}{2}$ scambi quando è ordinato in senso decrescente con chiavi distinte.

Dimostrazione (1/3)

- Nell'iterazione i del ciclo esterno, $i \in \{1, 2, \ldots, n-1\}$, il ciclo interno esegue la riga 6 una volta per ogni $j \in \{i+1, i+2, \ldots, n\}$, cioè $n - i$ volte
- Per la somma dei primi $n-1$ interi positivi, il numero totale di confronti tra chiavi è

$$\sum_{i=1}^{n-1}(n-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$$

- Ogni confronto provoca al più uno scambio, quindi gli scambi sono al più $\frac{n(n-1)}{2}$
- Se l'array è ordinato, $a[i] \leq a[j]$ per ogni $i < j$: il primo confronto non provoca alcuno scambio, l'array non cambia, e lo stesso vale per tutti i confronti successivi

Dimostrazione (2/3)

- Se le chiavi sono distinte e in ordine decrescente, sia $b[1..n]$ l'array di input, quindi $b[1] > b[2] > \cdots > b[n]$
- Nell'iterazione 1, prima del confronto con $j \in \{2, 3, \ldots, n\}$ il valore corrente di $a[1]$ è $b[j-1]$ (l'ultima chiave scambiata in posizione 1), che è maggiore di $a[j] = b[j]$
- Quindi ogni confronto provoca uno scambio, che porta $b[j-1]$ in posizione j

Dimostrazione (3/3)

- Alla fine dell'iterazione 1,

$$a[1] = b[n] \qquad \text{e} \qquad a[2..n] = [b[1], b[2], \ldots, b[n-1]],$$

che è di nuovo in ordine decrescente

- Ad esempio, $[6, 5, 4, 3, 2, 1]$ diventa $[1, 6, 5, 4, 3, 2]$
- Ripetendo il ragionamento su $a[i..n]$ per $i = 2, 3, \ldots, n-1$, tutti i $\frac{n(n-1)}{2}$ confronti provocano uno scambio

□

Gli scambi costano più degli spostamenti

- Su un array in ordine decrescente con chiavi distinte, Exchange Sort esegue $\frac{n(n-1)}{2}$ scambi, cioè

$$\frac{3n(n-1)}{2} \quad \text{assegnazioni di chiavi}$$

- Insertion Sort sposta le chiavi con $\frac{n(n-1)}{2}$ spostamenti, di una assegnazione ciascuno

Osservazione 4: Exchange Sort è in-place ma non stabile

`ExchangeSort` è un algoritmo di ordinamento in-place, ma non è stabile.

Dimostrazione (1/2)

- Oltre all'array, l'algoritmo usa soltanto le tre variabili i, j e tmp: è in-place
- Per mostrare che non è stabile basta esibire un input su cui l'ordine relativo di due chiavi uguali cambia
- Consideriamo l'array di record $[(2, A),\ (2, B),\ (1, C)]$

Dimostrazione (2/2)

- Per $i = 1$:
  - il confronto con $j = 2$ dà $2 > 2$ falso, quindi nessuno scambio;
  - il confronto con $j = 3$ dà $2 > 1$, quindi $a[1]$ e $a[3]$ vengono scambiati, ottenendo $[(1, C),\ (2, B),\ (2, A)]$
- Per $i = 2$: $2 > 2$ è falso, quindi nessuno scambio
- L'output è $[(1, C),\ (2, B),\ (2, A)]$: il record $(2, A)$, che nell'input precedeva $(2, B)$, ora lo segue

□

Dallo pseudocodice al codice

- Nelle implementazioni (C++, Python e Java, nei notebook Colab qui sopra) gli indici vanno da 0 a $n-1$
- Il ciclo esterno scorre con i da 0 a $n-2$, e il ciclo interno con j da $i+1$ a $n-1$

Riepilogo

Exchange Sort:

- Nell'iterazione i, confronta $a[i]$ con ogni $a[j]$, $j > i$, e scambia ogni volta che $a[i] > a[j]$: alla fine $a[i] = \min(a[i..n])$

Costo:

- Sempre esattamente $\frac{n(n-1)}{2}$ confronti tra chiavi
- Tra 0 (array ordinato) e $\frac{n(n-1)}{2}$ (array decrescente) scambi, di tre assegnazioni ciascuno

Proprietà:

- In-place, ma non stabile

Selection Sort

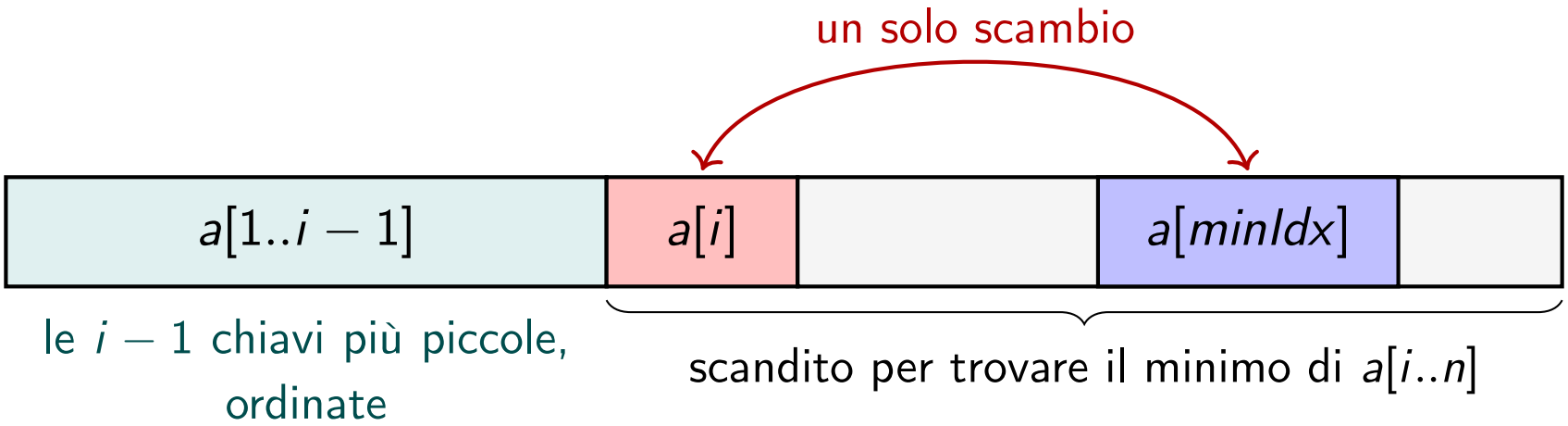
Introduzione: Selection Sort

- Il terzo algoritmo ottiene lo stesso risultato di Exchange Sort con molti meno scambi
- Diamo il suo pseudocodice e lo eseguiamo sugli stessi due array
- Vedremo che esegue al più $n - 1$ scambi, il minor numero di assegnazioni di chiavi nel caso peggiore tra i tre algoritmi, e che non è stabile

L'idea di Selection Sort

- Exchange Sort può scambiare molte volte nella stessa iterazione solo per portare il minimo di $a[i..n]$ nella posizione i
- Selection Sort ottiene lo stesso risultato con al più uno scambio per iterazione:
  - prima *seleziona* la posizione del minimo di $a[i..n]$,
  - e poi lo scambia con $a[i]$

Iterazione i di Selection Sort



- Il ciclo interno trova la posizione *minIdx* del minimo di $a[i..n]$, e poi $a[i]$ e $a[minIdx]$ vengono scambiati (se $minIdx \neq i$)

Algorithm: SelectionSort (1/2)

Input : Un array $a[1..n]$ di n numeri naturali
Output : L'array a ordinato in ordine non decrescente

```
1  function SelectionSort(a, n)
2      i := 1
3      while i ≤ n − 1 do
4          minIdx := i          // indice del minimo in a[i..n]
5          j := i + 1
6          while j ≤ n do
7              if a[j] < a[minIdx] then
8                  minIdx ← j   // trovato un nuovo minimo
9              end
10             j ← j + 1
11         end
```

Algorithm: SelectionSort (2/2)

```
function SelectionSort(a, n) (continua)
    while i ≤ n − 1 (continua) do
12      if minIdx ≠ i then
13          tmp := a[i]          // scambia a[i] e a[minIdx]
14          a[i] ← a[minIdx]
15          a[minIdx] ← tmp
16      end
17      i ← i + 1
18  end
19 end
```

Esegui in Colab: Python · C++ · Java

Come funziona SelectionSort

- Il ciclo esterno scorre con l'indice i da 1 a $n - 1$
- Per ogni i, il ciclo interno scandisce $a[i + 1..n]$ e mantiene in *minIdx* l'indice della più piccola chiave incontrata finora
- Poiché il confronto alla riga 7 è stretto, in caso di parità *minIdx* è l'indice del *primo* minimo
- Dopo il ciclo interno, se $minIdx \neq i$, il minimo $a[minIdx]$ viene scambiato con $a[i]$: al più **uno scambio** per iterazione del ciclo esterno, a differenza di Exchange Sort
- Dopo l'iterazione i del ciclo esterno, il sotto-array $a[1..i]$ è ordinato e contiene le i chiavi più piccole dell'array

Esempio: Selection Sort su $[5, 2, 4, 1, 3]$ (1/3)

Array di input: $a = [5, 2, 4, 1, 3]$, con $n = 5$. Il ciclo esterno esegue le iterazioni $i = 1, 2, 3, 4$.

Traccia dettagliata (ciclo interno), iterazione $i = 1$:

i	j	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	–	–	5	2	4	1	3	stato iniziale
1	–	1	5	2	4	1	3	$minIdx := 1$ ($a[1] = 5$)
	2	2	5	2	4	1	3	$a[2] = 2 < a[1] = 5$: $minIdx \leftarrow 2$
	3	2	5	2	4	1	3	$a[3] = 4 \geq a[2] = 2$: nessun aggiornamento
	4	4	5	2	4	1	3	$a[4] = 1 < a[2] = 2$: $minIdx \leftarrow 4$
	5	4	5	2	4	1	3	$a[5] = 3 \geq a[4] = 1$: nessun aggiornamento
	–	–	1	2	4	5	3	$minIdx = 4 \neq 1$: scambia $a[1] \leftrightarrow a[4]$

Esempio: Selection Sort su $[5, 2, 4, 1, 3]$ (2/3)

Traccia dettagliata (ciclo interno), iterazioni $i = 2$, $i = 3$ e $i = 4$:

i	j	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
2	–	2	1	2	4	5	3	$minIdx := 2$ ($a[2] = 2$)
	3	2	1	2	4	5	3	$a[3] = 4 \geq a[2] = 2$: nessun aggiornamento
	4	2	1	2	4	5	3	$a[4] = 5 \geq a[2] = 2$: nessun aggiornamento
	5	2	1	2	4	5	3	$a[5] = 3 \geq a[2] = 2$: nessun aggiornamento
	–	–	1	2	4	5	3	$minIdx = 2$: nessuno scambio
3	–	3	1	2	4	5	3	$minIdx := 3$ ($a[3] = 4$)
	4	3	1	2	4	5	3	$a[4] = 5 \geq a[3] = 4$: nessun aggiornamento
	5	5	1	2	4	5	3	$a[5] = 3 < a[3] = 4$: $minIdx \leftarrow 5$
	–	–	1	2	3	5	4	$minIdx = 5 \neq 3$: scambia $a[3] \leftrightarrow a[5]$
4	–	4	1	2	3	5	4	$minIdx := 4$ ($a[4] = 5$)
	5	5	1	2	3	5	4	$a[5] = 4 < a[4] = 5$: $minIdx \leftarrow 5$
	–	–	1	2	3	4	5	$minIdx = 5 \neq 4$: scambia $a[4] \leftrightarrow a[5]$

Esempio: Selection Sort su $[5, 2, 4, 1, 3]$ (3/3)

Stato dell'array dopo ogni iterazione del ciclo esterno:

i	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	–	5	2	4	1	3	stato iniziale
1	4	1	2	4	5	3	scambia $a[1] \leftrightarrow a[4]$: $a[1] = \min(a[1..5]) = 1$
2	2	1	2	4	5	3	nessuno scambio: $a[2] = \min(a[2..5]) = 2$
3	5	1	2	3	5	4	scambia $a[3] \leftrightarrow a[5]$: $a[3] = \min(a[3..5]) = 3$
4	5	1	2	3	4	5	scambia $a[4] \leftrightarrow a[5]$: $a[4] = \min(a[4..5]) = 4$

- L'array ordinato è $[1, 2, 3, 4, 5]$
- L'algoritmo esegue 10 confronti tra chiavi e 3 scambi

Esempio: Selection Sort su $[12, 11, 13, 5, 6]$ (1/3)

Array di input: $a = [12, 11, 13, 5, 6]$, con $n = 5$.

Traccia dettagliata (ciclo interno), iterazioni $i = 1$ e $i = 2$:

i	j	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	–	–	12	11	13	5	6	stato iniziale
1	–	1	12	11	13	5	6	$minIdx := 1$ ($a[1] = 12$)
	2	2	12	11	13	5	6	$a[2] = 11 < a[1] = 12$: $minIdx \leftarrow 2$
	3	2	12	11	13	5	6	$a[3] = 13 \geq a[2] = 11$: nessun aggiornamento
	4	4	12	11	13	5	6	$a[4] = 5 < a[2] = 11$: $minIdx \leftarrow 4$
	5	4	12	11	13	5	6	$a[5] = 6 \geq a[4] = 5$: nessun aggiornamento
	–	–	5	11	13	12	6	$minIdx = 4 \neq 1$: scambia $a[1] \leftrightarrow a[4]$
2	–	2	5	11	13	12	6	$minIdx := 2$ ($a[2] = 11$)
	3	2	5	11	13	12	6	$a[3] = 13 \geq a[2] = 11$: nessun aggiornamento
	4	2	5	11	13	12	6	$a[4] = 12 \geq a[2] = 11$: nessun aggiornamento
	5	5	5	11	13	12	6	$a[5] = 6 < a[2] = 11$: $minIdx \leftarrow 5$
	–	–	5	6	13	12	11	$minIdx = 5 \neq 2$: scambia $a[2] \leftrightarrow a[5]$

Esempio: Selection Sort su $[12, 11, 13, 5, 6]$ (2/3)

Traccia dettagliata (ciclo interno), iterazioni $i = 3$ e $i = 4$:

i	j	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
3	–	3	5	6	13	12	11	$minIdx := 3$ ($a[3] = 13$)
	4	4	5	6	13	12	11	$a[4] = 12 < a[3] = 13$: $minIdx \leftarrow 4$
	5	5	5	6	13	12	11	$a[5] = 11 < a[4] = 12$: $minIdx \leftarrow 5$
	–	–	5	6	11	12	13	$minIdx = 5 \neq 3$: scambia $a[3] \leftrightarrow a[5]$
4	–	4	5	6	11	12	13	$minIdx := 4$ ($a[4] = 12$)
	5	4	5	6	11	12	13	$a[5] = 13 \geq a[4] = 12$: nessun aggiornamento
	–	–	5	6	11	12	13	$minIdx = 4$: nessuno scambio

Esempio: Selection Sort su $[12, 11, 13, 5, 6]$ (3/3)

Stato dell'array dopo ogni iterazione del ciclo esterno:

i	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Azione
–	–	12	11	13	5	6	stato iniziale
1	4	5	11	13	12	6	scambia $a[1] \leftrightarrow a[4]$: $a[1] = \min(a[1..5]) = 5$
2	5	5	6	13	12	11	scambia $a[2] \leftrightarrow a[5]$: $a[2] = \min(a[2..5]) = 6$
3	5	5	6	11	12	13	scambia $a[3] \leftrightarrow a[5]$: $a[3] = \min(a[3..5]) = 11$
4	4	5	6	11	12	13	nessuno scambio: $a[4] = \min(a[4..5]) = 12$

- L'array ordinato è $[5, 6, 11, 12, 13]$
- L'algoritmo esegue 10 confronti tra chiavi e 3 scambi

Contiamo ora i confronti e gli scambi di Selection Sort,
e verifichiamo se è in-place e stabile.

Osservazione 5: confronti e scambi di Selection Sort

Dato un array $a[1..n]$ *di* n *numeri naturali,* `SelectionSort` *esegue esattamente* $\frac{n(n-1)}{2}$ *confronti tra chiavi e al più* $n-1$ *scambi. Non viene eseguito alcuno scambio quando l'array è già ordinato, e vengono eseguiti* $n-1$ *scambi, ad esempio, sull'array* $[2, 3, \ldots, n, 1]$.

Dimostrazione (1/2)

- Come per Exchange Sort, nell'iterazione $i \in \{1, 2, \ldots, n-1\}$ il confronto alla riga 7 viene eseguito una volta per ogni $j \in \{i+1, i+2, \ldots, n\}$, quindi il numero totale di confronti tra chiavi è

$$\sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2}$$

  per la somma dei primi $n-1$ interi positivi
- Ciascuna delle $n-1$ iterazioni del ciclo esterno esegue al più uno scambio (righe 13–15), quindi gli scambi sono al più $n-1$
- Se l'array è ordinato, il minimo di $a[i..n]$ è $a[i]$ e, per il confronto stretto alla riga 7, *minIdx* resta uguale a i: non viene eseguito alcuno scambio

Dimostrazione (2/2)

- Su $[2, 3, \ldots, n, 1]$, l'iterazione 1 trova il minimo 1 in posizione n e lo scambia con $a[1] = 2$, ottenendo $[1, 3, 4, \ldots, n, 2]$
- In generale, all'inizio dell'iterazione i l'array è

$$[1, 2, \ldots, i-1, i+1, \ldots, n, i],$$

  il minimo i di $a[i..n]$ è in posizione $n \neq i$, e lo scambio produce

$$[1, 2, \ldots, i, i+2, \ldots, n, i+1]$$

- Dunque ciascuna delle $n-1$ iterazioni esegue uno scambio

□

Poche assegnazioni, confronti fissi

- Selection Sort esegue al più $3(n-1)$ assegnazioni di chiavi
- È il minor numero di assegnazioni di chiavi nel caso peggiore tra i tre algoritmi di questo capitolo
- Il suo numero di confronti, $\frac{n(n-1)}{2}$, non dipende dall'input

Osservazione 6: Selection Sort è in-place ma non stabile

`SelectionSort` è un algoritmo di ordinamento in-place, ma non è stabile.

Dimostrazione

- Oltre all'array, l'algoritmo usa soltanto le quattro variabili i, j, *minIdx* e *tmp*: è in-place
- Consideriamo l'array di record $[(2, A),\ (2, B),\ (1, C)]$
- Per $i = 1$, il minimo è $(1, C)$, con $minIdx = 3$, e lo scambio di $a[1]$ e $a[3]$ dà $[(1, C),\ (2, B),\ (2, A)]$
- Per $i = 2$, il confronto $2 < 2$ è falso, quindi $minIdx = 2$ e non viene eseguito alcuno scambio
- Nell'output $(2, A)$ segue $(2, B)$, mentre nell'input lo precedeva: lo scambio ha spostato $(2, A)$ oltre la chiave uguale $(2, B)$

□

Dallo pseudocodice al codice

- Nelle implementazioni (C++, Python e Java, nei notebook Colab qui sopra) gli indici vanno da 0 a $n-1$
- Il ciclo esterno scorre con i da 0 a $n-2$, e *minIdx* parte da i; il ciclo interno scorre con j da $i+1$ a $n-1$

Riepilogo

Selection Sort:

- Nell'iterazione i, seleziona la posizione *minIdx* del minimo di $a[i..n]$, poi scambia $a[i]$ e $a[minIdx]$ (se $minIdx \neq i$)

Costo:

- Sempre esattamente $\frac{n(n-1)}{2}$ confronti tra chiavi
- Al più $n-1$ scambi, cioè al più $3(n-1)$ assegnazioni di chiavi

Proprietà:

- In-place, ma non stabile

Confronto dei tre algoritmi

Introduzione: Confronto dei tre algoritmi

- Mettiamo a confronto i risultati delle sei Osservazioni del capitolo
- Confrontiamo i tre algoritmi sui due array usati in tutto il capitolo
- Il confronto ci dice quale algoritmo scegliere a seconda di che cosa conta: i confronti, gli spostamenti di chiavi o la stabilità

I tre algoritmi a confronto

- La tabella raccoglie i risultati delle sei Osservazioni di questo capitolo, per un array di n chiavi
- Gli "spostamenti" sono gli spostamenti (shift) per Insertion Sort e gli scambi (swap) per gli altri due algoritmi

	confronti tra chiavi		spostamenti			
algoritmo	migliore	peggiore	migliore	peggiore	in-place	stabile
Insertion Sort	$n-1$	$\frac{n(n-1)}{2}$	0 shift	$\frac{n(n-1)}{2}$ shift	sì	sì
Exchange Sort	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	0 scambi	$\frac{n(n-1)}{2}$ scambi	sì	no
Selection Sort	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	0 scambi	$n-1$ scambi	sì	no

Esempio

I tre algoritmi sugli stessi array

Sui due array usati in tutto il capitolo, i tre algoritmi eseguono i seguenti numeri di confronti tra chiavi e di spostamenti.

	$[5, 2, 4, 1, 3]$		$[12, 11, 13, 5, 6]$	
algoritmo	confronti	spostamenti	confronti	spostamenti
Insertion Sort	9	7 shift	9	7 shift
Exchange Sort	10	7 scambi	10	7 scambi
Selection Sort	10	3 scambi	10	3 scambi

Quale algoritmo?

- Insertion Sort, Exchange Sort e Selection Sort sono basati su confronti e in-place, e nel caso peggiore tutti e tre eseguono $\frac{n(n-1)}{2}$ confronti tra chiavi
- Insertion Sort si adatta all'input: su un array già ordinato esegue soltanto $n-1$ confronti e nessuno spostamento. È l'unico algoritmo stabile dei tre
- Exchange Sort e Selection Sort eseguono sempre $\frac{n(n-1)}{2}$ confronti, ma Selection Sort esegue al più $n-1$ scambi, contro i $\frac{n(n-1)}{2}$ scambi di Exchange Sort nel caso peggiore

Riepilogo (1/2)

Il problema dell'ordinamento:

- Riarrangiare le chiavi di $a[1..n]$ in ordine non decrescente
- Gli algoritmi basati su confronti soltanto confrontano e copiano le chiavi; contiamo i confronti tra chiavi e le assegnazioni di chiavi nel caso migliore e peggiore, la memoria aggiuntiva e la stabilità

Insertion Sort:

- Inserisce ogni chiave nel prefisso ordinato, spostando a destra le chiavi maggiori
- Da $n-1$ a $\frac{n(n-1)}{2}$ confronti, al più $\frac{n(n-1)}{2}$ spostamenti; in-place e stabile

Riepilogo (2/2)

Exchange Sort:

- Porta il minimo di $a[i..n]$ nella posizione i con scambi ripetuti
- Esattamente $\frac{n(n-1)}{2}$ confronti, al più $\frac{n(n-1)}{2}$ scambi; in-place, non stabile

Selection Sort:

- Seleziona il minimo di $a[i..n]$ e lo porta con un solo scambio nella posizione i
- Esattamente $\frac{n(n-1)}{2}$ confronti, al più $n-1$ scambi; in-place, non stabile

Scelta: Insertion Sort si adatta all'input ed è stabile; Selection Sort sposta le chiavi meno di tutti